

iSAQB Module: Formal Methods - Specification as Code

Active Group GmbH

Created: 2026-10-06 Tue 09:34

Exercise: Vending Machine Algorithm

How would you specify the algorithm for a simple vending machine?

- It has two buttons for selecting tea or coffee.
- It has a slot for inserting a coin.
- Three coins will buy a drink.
- All coins beyond three coins will be returned.

Different Forms of Specification

- natural-language description vs.
 - separate mathematical formulae vs.
 - mathematical formulae as code
-
- individual examples vs.
 - general, quantified rules

Vending Machine in C

```
typedef unsigned long money_t;  
typedef enum output_e {  
    NO_OUTPUT, DISPENSE_TEA, DISPENSE_COFFEE, RETURN_COIN  
} output_t;  
typedef enum selection_e {  
    NOTHING, TEA, COFFEE  
} selection_t;  
  
int money = 0;  
selection_t selection = NOTHING;  
  
const int product_price = 3;
```

Vending Machine in C

```
int
main(void)
{
    char *result;
    char line[MAX_LINE_LENGTH];

    while ((result = fgets(line, MAX_LINE_LENGTH, stdin)) != NULL) {
        output_t output = NO_OUTPUT;
        if (strncmp("tea", line, 3) == 0)
            output = select(TEA);
        else if (strncmp("coffee", line, 6) == 0)
            output = select(COFFEE);
        else if (strncmp("coin", line, 4) == 0) {
            output = insert_coin();
        }
        print(output);
    }
}
```

Vending Machine in C

```
output_t  
maybeDispense(void) { ... }
```

```
output_t  
insert_coin(void) { ... }
```

```
output_t  
select(selection_t new_selection) { ... }
```

```
void  
print(output_t output) { ... }
```

Aside: Static Analysis to Predict Dynamic Behavior

Frama-C generates the following output for the vending machine:

```
eva:final-states] Values at end of function maybeDispense:
  money ∈ [-2147483648..3]
  selection ∈ [--..--]
  __retres ∈ {NO_OUTPUT; DISPENSE_TEA; DISPENSE_COFFEE}
[eva:final-states] Values at end of function insert_coin:
  money ∈ [-2147483647..2147483647]
  selection ∈ [--..--]
  __retres ∈ {NO_OUTPUT; DISPENSE_TEA; DISPENSE_COFFEE; RETURN_COIN}
[eva:final-states] Values at end of function select:
  money ∈ [-2147483648..2]
  selection ∈ {NOTHING; TEA; COFFEE}
```

QuickCheck vs. Mersenne Conjecture

$$\forall n. \text{prime}(n) \Rightarrow \text{prime}(2^n - 1)$$

```
divides :: Integer -> Integer -> Bool  
m `divides` n = n `mod` m == 0
```

```
isPrime :: Integer -> Bool  
isPrime n | n <= 1 = False  
isPrime 2 = True  
isPrime n = not (any (`divides` n) [2..n-1])
```

```
quickCheck(\n -> isPrime n ==> isPrime(2^n - 1))
```

Matt Might: An introduction to QuickCheck by example

Model-Based Testing

```
data State =  
  MkState (Maybe Product) -- we only track what product was selected  
  deriving Show  
  
instance StateModel State where  
  data Action State  
    = Select Product  
    | InsertCoin  
    deriving Show  
  
data Ret State = Dispense Product | GiveBackCoin | Noop  
  deriving Show  
  
initialState :: State  
initialState = MkState Nothing  
  
nextState :: State -> Action State -> Step -> State  
nextState (MkState _) (Select product) _step =  
  MkState (Just product)  
nextState state@(MkState selection) InsertCoin _step =  
  state
```

Postcondition

```
postcondition :: State -> Action State -> (Step -> Ret State) -> Ret State -> Bool
postcondition (MkState _) (Select product) _getStep (Dispense product') =
    product == product'
postcondition (MkState _) (Select product) _getStep GiveBackCoin = False
postcondition (MkState _) (Select product) _getStep Noop = True

postcondition (MkState _) InsertCoin _getStep GiveBackCoin = True
postcondition (MkState _) InsertCoin _getStep Noop = True
-- something for nothing
postcondition (MkState Nothing) InsertCoin _getStep (Dispense product) = False
postcondition (MkState (Just product)) InsertCoin _getStep (Dispense product') =
    product == product'
```

QuickChecking a StateModel

```
instance StateModel State where
  ...
  type ActionMonad State = I0

  arbitraryAction :: State -> Gen (Action State)
  arbitraryAction _state =
    oneof [Select <$> elements [Tea, Coffee],
          pure InsertCoin]
  ...

prop_Vending :: Script State -> Property
prop_Vending s = monadicIO $ do
  run startCVending
  runScript s
  run finishCVending
  assert True

main = quickCheck prop_Vending
```

Exercise: Bugs vs. QuickCheck

What kinds of bugs can QuickCheck test for the vending machine, and for which ones does it fail? Experiment!

Formulate a property that you can't specify with QuickCheck!

Exercise (optional): Pacemaker Level Implementation

Write a C program for the pacemaker component that determines the magnitude of the stimulus.

Exercise: Pacemaker Level Testing

Write QuickCheck tests that express the requirements of the pacemaker component, and check your C implementation against it.

Forge: A Tool and Language for Teaching Formal Methods

- declarative language based on logic
- *model finder*: serves as vehicle to explore model checking
- supports both purely relational specifications and LTL

Forge/Froglet

```
#lang forge/froglet

abstract sig Input {}
one sig Coin, SelectTea, SelectCoffee extends Input {}

abstract sig Output {}
one sig ReturnCoin, DispenseTea, DispenseCoffee extends Output {}

abstract sig Selection {}
one sig Tea, Coffee extends Selection {}
```

Vending machine state

```
sig State {  
  money : one Int,  
  selection : lone Selection,  
  output : lone Output, -- output before we got here  
  next : lone State  
}
```

Field Multiplicities

- one singleton value
- lone either singleton value or no value
- `func A -> B -> ... -> Y -> Z` total function
- `pfunc A -> B -> ... -> Y -> Z` partial function

Later:

- set set of values

Trace of vending-machine execution

```
one sig Trace {  
    initialState : one State,  
    inputAfter : pfunc State -> Input  
}
```

State transitions

```
pred machine_transition[prevState: State,  
                        input: Input,  
                        nextState: State ] {  
    -- see code  
}
```

Formulas

- Negation: not (!)
- Conjunction: and (&&)
- Disjunction: or (| |)
- Implication: implies ($\Rightarrow$)
- If-then-else: else
- If-and-only-if: iff ($\Leftrightarrow$)

Cardinality and Membership

- `no <expr>`: true when `<expr>` is empty
- `lone <expr>`: true when `<expr>` contains zero or one elements
- `one <expr>`: true when `<expr>` contains exactly one element
- `some <expr>`: true when `<expr>` contains at least one element
- `<expr-a> in <expr-b>`: true when `<expr-a>` is a subset of or equal to `<expr-b>`
- `<expr-a> = <expr-b>`: true when `<expr-a>` and `<expr-b>` contain exactly the same elements

Exercise: Relation to Algorithm

What additional specification is necessary to turn the transition relation into an algorithm - i.e. to specify the next field in State instances?

Reachability

`reachable[a, b, f]`

Object `a` is reachable from `b` through recursively applying field `f` one or more times.

Quantifiers

- $\text{some } \langle x \rangle : \langle \text{expr} \rangle \mid \{ \langle \text{fm} \text{la} \rangle \}$: true when $\langle \text{fm} \text{la} \rangle$ is true for at least one element in $\langle \text{expr} \rangle$; and
- $\text{all } \langle x \rangle : \langle \text{expr} \rangle \mid \{ \langle \text{fm} \text{la} \rangle \}$: true when $\langle \text{fm} \text{la} \rangle$ is true for all elements in $\langle \text{expr} \rangle$

Several variables:

- $\text{some } \langle x \rangle : \langle \text{expr-a} \rangle, \langle y \rangle : \langle \text{expr-b} \rangle \mid \{ \langle \text{fm} \text{la} \rangle \}$
- $\text{some } \langle x \rangle, \langle y \rangle : \langle \text{expr} \rangle \mid \{ \langle \text{fm} \text{la} \rangle \}$

Disjointness:

- $\text{some disj } x, y : A \mid \dots$
- $\text{all disj } x, y : A \mid \dots$

State Transition to Algorithm

```
pred algorithm {  
  // initial state  
  Trace.initialState.money = 0  
  no Trace.initialState.selection  
  no Trace.initialState.output  
  
  // trace follows state transition  
  all a, b: State {  
    a.next = b => {  
      some i: Input | {  
        Trace.inputAfter[a] = i and machine_transition[a, i, b]}}}  
  
  // all states are reachable from initial state  
  all a: State | { a = Trace.initialState or reachable[a, Trace.initialState, next}  
  
  // transition doesn't loop  
  no a: State | { a != Trace.initialState and reachable[Trace.initialState, a, next]  
}
```

Example

| example ex1 is algorithm for { ... }

Exercise: Forge Output

Run example ex1 in Forge and understand the output in both graph and table format! Experiment with over- and underconstraining the example.

Forge Running/Testing

- run: find instances that satisfy a predicate
- check: find counterexample (UNSAT is good, actually)
- example: specific instance satisfies a predicate
- assert: + necessary / sufficient

We Can Get Tea in 4 Steps

```
tea_in_4_steps: run {  
  algorithm  
    some s1, s2, s3, s4, s5: State, i1, i2, i3, i4: Input | {  
      Trace.inputAfter[s1] = i1  
      Trace.inputAfter[s2] = i2  
      Trace.inputAfter[s3] = i3  
      Trace.inputAfter[s4] = i4  
      Trace.initialState = s1  
      s1.next = s2  
      s2.next = s3  
      s3.next = s4  
      s4.next = s5  
      s5.output = DispenseTea  
    }  
  } for 5 State for { next is linear }
```

Relational Modeling

- tedious
- underlying domain is time-based

Temporal Forge

```
#lang forge/temporal

abstract sig Input {} ...
abstract sig Output {} ...
abstract sig Selection {} ...
```

Variables

```
sig State {  
  money : one Int,  
  selection : lone Selection,  
  output : lone Output -- output before we got here  
}  
  
one sig Trace {  
  var currentState : one State,  
  var input : one Input  
}
```

State Transition

```
pred machine_transition {  
  Trace.currentState.money = 3 => {  
    Trace.input = Coin => {  
      Trace.currentState'.money = 3  
      Trace.currentState'.selection = Trace.currentState.selection  
      Trace.currentState'.output = ReturnCoin  
    }  
    Trace.input = SelectTea => {  
      Trace.currentState'.money = 0  
      no Trace.currentState'.selection  
      Trace.currentState'.output = DispenseTea  
    }  
  }  
}  
...
```

Algorithm

```
pred algorithm {  
    // initial state  
    Trace.currentState.money = 0  
    no Trace.currentState.selection  
    no Trace.currentState.output  
    always machine_transition  
}
```

Temporal Operators

- `next_state <fmla>` is true in a state i if and only if: `fmla` holds in state $i+1$.
- `always <fmla>` is true in a state i if and only if: `fmla` holds in every state $\geq i$.
- `eventually <fmla>` is true in a state i if and only if: `fmla` holds in some state $\geq i$.
- `<fmla-a> until <fmla-b>` is true in a state i if and only if: `fmla-b` holds in some state $j \geq i$ and `fmla-a` holds in all states k where $i < k < j$. Note that the obligation to see `fmla-a` ceases in the state where `fmla-b` holds.

Tea in 4 Steps

```
always_tea_in_4_steps: check {
  algorithm => {
    always {
      some i1, i2, i3, i4: Input | {
        Trace.input = i1
        next_state Trace.input = i2
        next_state next_state Trace.input = i3
        next_state next_state next_state Trace.input = i4
      } => next_state next_state next_state next_state Trace.currentState.output = Disp
    }
  }
} for 10 State
```

Exercise: Pacemaker Level Specification

Design an appropriate Forge interface for the pacemaker component that determines the magnitude of the stimulus, along with predicates that check the requirements of that component.

Choose relational Forge or temporal Forge as appropriate.

Exercise: Pacemaker Implementation

Provide a relational implementation of the pacemaker level algorithm, and verify it against the requirements from the previous exercise.

Possible Pacemaker Refinement

Relational Specification

→ Haskell Implementation

→ C Implementation